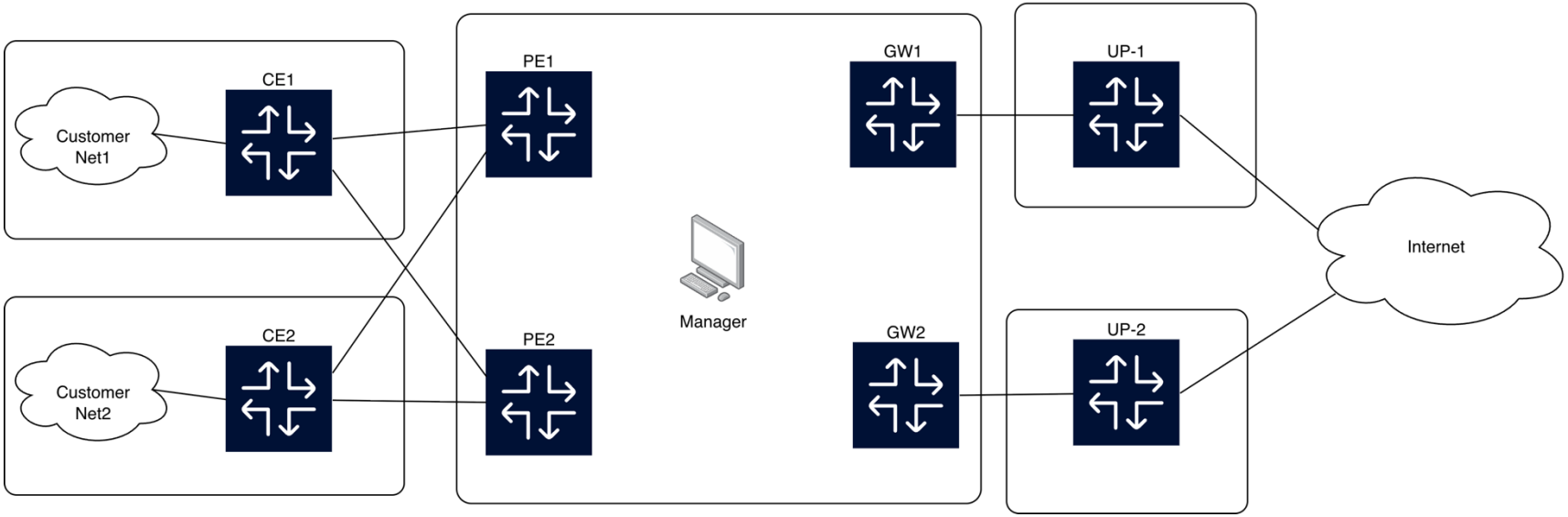
Advanced Computer Networking Group Project

UNIVERSITÀ DI PISA

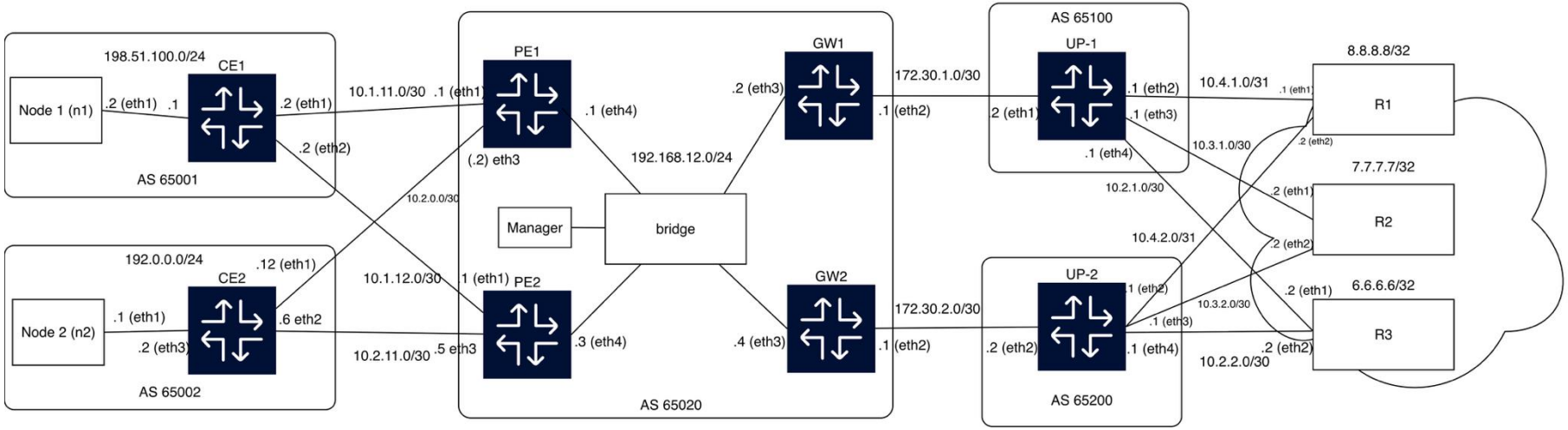
Authors
Valerio Triolo
Luca Arduini

Specification



- Task 1: Configuration Templates with Jinja2
- Task 2: Network Emulation with Containerlab and FRR
- Task 3: Network Automation System for AS65020

Configured architecture



- Configured all the interfaces and assigned all IP addresses
- Introduced Node 1 and Node 2 as customer nodes
- Represented internet destinations as three routers (R1, R2, R3)

Task 1

- Data model (data.yaml)
  - List of nodes and interfaces
  - Links between nodes

- Script to the generate configs (build_configs.py)
  - Read data.yaml and output a .conf file
  - Automate iBGP/eBGP config

- Config template (config.j2)

  Used Jinja2 as template engine

  To parametrize the config file

Task 1 - Details

```yaml
nodes:
  - name: n1
    role: host
    ipv4_address: 198.51.100.3
    ipv4_mask: /24
    interfaces:
      - { name: eth1, ipv4_address: "198.51.100.2", ipv4_mask: /24 }
    gateway_ip: 198.51.100.1

  - name: n2
    role: host
    ipv4_address: 192.0.0.3
    ipv4_mask: /24
    interfaces:
      - { name: eth1, ipv4_address: "192.0.0.1", ipv4_mask: /24 }
    gateway_ip: 192.0.0.2

  - name: ce1
    role: ce
    bgp: { asn: 65001, router_id: 10.0.0.1 }
    interfaces:
      - { name: eth3, ipv4_address: "198.51.100.1", ipv4_mask: /24 }
      - { name: eth1, ipv4_address: "10.1.11.1", ipv4_mask: /30 }
      - { name: eth2, ipv4_address: "10.1.12.2", ipv4_mask: /30 }

  - name: ce2
    role: ce
    bgp: { asn: 65002, router_id: 10.0.0.2 }
    interfaces:
      - { name: eth3, ipv4_address: "192.0.0.2", ipv4_mask: /24 }
      - { name: eth1, ipv4_address: "10.2.0.1", ipv4_mask: /30 }
      - { name: eth2, ipv4_address: "10.2.11.6", ipv4_mask: /30 }

  - name: pe1
    role: pe
```

es > config.j2

```jinja2
frr version 10.0.2_git
frr defaults traditional

hostname {{ device.name }}

{% for intf in interfaces %}
interface {{ intf.name }}
    {% if intf.ipv4_address %}
    ip address {{ intf.ipv4_address }}{{ intf.ipv4_mask }}
    {% endif %}
exit
!
{% endfor %}

{% if device.bgp %}
router bgp {{ device.bgp.asn }}
    bgp router-id {{ device.bgp.router_id }}
    no bgp ebgp-requires-policy

    {% for net in networks %}
    network {{ net }}
    {% endfor %}

    {% for neighbor in neighbors %}
    neighbor {{ neighbor.ip }} remote-as {{ neighbor.remote_as }}
        {% if neighbor.type == 'ibgp' %}
        neighbor {{ neighbor.ip }} next-hop-self
        {% endif %}
        {% if neighbor.policy and neighbor.policy.type == 'set_local_pref' %}
        neighbor {{ neighbor.ip }} route-map RM_LP_IN in
        {% endif %}
    {% endfor %}
exit
!
route-map RM_LP_IN permit 10
    set local-preference 200
!
{% endif %}
```

Task 2

- Data model (data.yaml)
  - List of nodes and interfaces
  - Links between nodes

- Generate the containerlab yaml file (build_topology.py)
- Used Jinja2 as Template Engine to parametrize the creation of the container for each node

Task 2 - Insights

- Used FRR Routing for routing device capabilities
  - quay.io/frrouting/frr:10.4.1
  - Usage of the .conf file generated with Task 1

- Used Alpine Linux for customer nodes
  - alpine:latest
  - Configured default gateway using a bash script based on GATEWAY_IP

- Used a custom docker image for the manager node

Task 3 – Centralized Control Plane

- **The Manager Node:**
  - Centralized control plane acting as the "Brain" of AS65020.
  - Container with scientific libraries (`numpy`, `scipy`).
- **Traffic Matrix Generation:**
  - Simulates bandwidth demand (Mbps) for Source-Dest pairs.
  - **Sparsity Constraint:** Forces specific flows to zero to test robustness.
- **Decomposed Views:**
  - **Ingress View ($T_{CE \to PE}$):** Granular flow data for PE selection.
  - **Egress View ($T_{PE \to GW}$):** Aggregated flow data for Gateway selection based on link capacity.

Mathematical Optimization Model

- **Approach:** Mixed-Integer Linear Programming

- **Problem 1: Ingress Optimization (Load Balancing)**
  - **Goal:** Minimize load difference between PE1 and PE2.
  - **Constraint:** Binary variable x_i (Flow goes to PE1 or PE2).
  - **Formulation:** $\min t \quad \text{s.t.} \quad |S_1 - S_2| \leq t$

- **Problem 2: Egress Optimization (Capacity Aware)**
  - **Goal:** Minimize Maximum Link Saturation.
  - **Constraint:** Link Capacities (Cap_1, Cap_2) are asymmetric.
  - **Formulation:** $\min t \quad \text{s.t.} \quad \frac{L_{gw}}{Cap_{gw}} \leq t$

From Math to BGP (Actuation)

- **Ingress Control (MED):**
  - Steers incoming traffic from CEs.
  - **Preferred PE:** MED 100 / **Backup:** MED 200.
- **Egress Control (Local Preference):**
  - Selects best exit path towards Upstreams.
  - **Preferred GW:** Local Pref 200 / **Backup:** Local Pref 100.
- **Execution Mechanism:**
  - Manager connects via **Docker Socket**.
  - Injects route-maps directly into routers using vtysh.
  - Soft reconfiguration (`clear bgp … soft`) ensures no packet loss.

Pipeline I - Orchestration & Deployment

- **Artifact Generation (`orchestrator.py`):**
  - **Topology:** Converts data.yaml → network.clab.yml.
  - **Configs:** Renders Jinja2 templates → frr.conf per node.
- **Synchronization:**
  - Transfers artifacts, scripts, and tests to the VM via **SCP**.
  - Ensures runtime environment mirrors local development.
- **Bootstrapping (`bootstrap.sh`):**
  - **Build:** Creates custom `acn-manager` image (Alpine + Python libs).
  - **Deploy:** Launches topology with containerlab deploy.

Pipeline II - The Automation Loop (`manager.py`)

- **1. Traffic Generation:**
  - Produces synthetic traffic matrix with sparsity.
- **2. Data Ingestion:**
  - Parses topology and current matrix state.
- **3. Optimization:**
  - Executes **Ingress Optimizer** (PE Selection).
  - Executes **Egress Optimizer** (GW Selection).
- **4. Actuation:**
  - Applies BGP policies to PEs via Docker execution.
- **5. Reporting:**
  - Exports `final_routing_paths.json` for validation.

Manager.py: Step-by-Step Execution Workflow

```
student@student1-2025-amd:~/ACN_bgp-automation$ docker exec -it clab-project-manager sh
~ # python3 automation/manager.py

========== STEP 1: Traffic Generation ==========
--- Topology Discovery ---
Source Routers (CE):      ['ce1', 'ce2']
PE Routers:               ['pe1', 'pe2']
Destination Routers:      ['r1', 'r2', 'r3']

--- Generating Traffic Matrix (Source -> Destination) ---

-----------------------------------------------------
Source / Destination |       r1 |       r2 |       r3
-----------------------------------------------------
ce1                  |        0 |       16 |       11 |
ce2                  |       39 |       40 |        0 |
-----------------------------------------------------

TRAFFIC POLICY: 1 zero per row, others 1-100 Mbps.

[OK] Traffic matrix saved to: /root/automation/traffic_matrix.json

========== STEP 2: Data Acquisition ==========

========== STEP 3: Optimization CE -> PE ==========
[DEBUG] 1D CE-PE Vector:
[0 1 2 1 2 0]

CE -> DESTINATION            | ASSIGNED PE
ce1          -> r2           | PE 1
ce1          -> r3           | PE 2
ce2          -> r1           | PE 1
ce2          -> r2           | PE 2

========== STEP 4: BGP MED Configuration (PE Selection) ==========
CE (Flow Source)     | DESTINATION     | Node (PE)           | MED
ce1                  | r2              | pe1                 | 100
ce1                  | r2              | pe2                 | 200
-------------------------------------------------------------------
ce1                  | r3              | pe1                 | 200
ce1                  | r3              | pe2                 | 100
-------------------------------------------------------------------
ce2                  | r1              | pe1                 | 100
ce2                  | r1              | pe2                 | 200
-------------------------------------------------------------------
ce2                  | r2              | pe1                 | 200
ce2                  | r2              | pe2                 | 100
-------------------------------------------------------------------
```

```
========== STEP 5: Aggregated Load Matrix (PE -> DEST) ==========
PE / DEST    |       r1 |       r2 |       r3
-----------------------------------------------
pe1          |       39 |       16 |        0
pe2          |        0 |       40 |       11

========== STEP 6: PE -> GW Optimization (Minimax Saturation) ==========
[INFO] Capacities loaded from YAML: GW1=1000.0, GW2=1000.0
[DEBUG] 1D PE-GW Vector:
[1 1 0 0 2 2]

PE -> DESTINATION            | ASSIGNED GW
pe1          -> r1           | GW 1
pe1          -> r2           | GW 1
pe2          -> r2           | GW 2
pe2          -> r3           | GW 2

========== STEP 7: BGP Local Preference Configuration (GW Selection) ====
PE (Flow Source)     | DESTINATION     | Node (GW)           | LOCAL PREF
pe1                  | r1              | gw1                 | 200
pe1                  | r1              | gw2                 | 100
-------------------------------------------------------------------
pe1                  | r2              | gw1                 | 200
pe1                  | r2              | gw2                 | 100
-------------------------------------------------------------------
pe2                  | r2              | gw1                 | 100
pe2                  | r2              | gw2                 | 200
-------------------------------------------------------------------
pe2                  | r3              | gw1                 | 100
pe2                  | r3              | gw2                 | 200
-------------------------------------------------------------------

========== STEP 8: End-to-End Paths Export ==========
[OK] Final JSON saved: /root/automation/final_routing_paths.json
```

Testing & Validation

- **Data Plane Validation (pings.py):**
  - Verifies End-to-End connectivity.
  - Uses ping -I (Source Interface) to validate full return paths via BGP.
- **Traffic Engineering Verification (te.py):**
  - Compares **Mathematical Decision** vs **Actual Path**.
  - Performs automated **traceroute** from containers.
  - **Link-Level Resolution:** Resolves physical interface IPs to confirm exact hops.
- **Result:**
  - Verified that BGP policies correctly steer packets according to the optimization model.

Thanks

For the attention